Augmenting the Cumulative Overload Check with Integral Resource Usage Reasoning

Samuel Cloutier
Université Laval, Québec, Canada

Claude-Guy Quimper
Université Laval, Québec, Canada

Abstract

Few amongst the rules that filter the Cumulative constraint reason on the *integrity* of resource consumption. We augment the overload check with an integral reasoning that better evaluates resource availability. This reasoning is based on the knapsack problem, which we solve using dynamic programming. Our new rule subsumes the time table horizontally elastic overload check while its running time complexity is $\mathcal{O}(Cn^2)$, only increasing by a factor C (the resource capacity) due to solving knapsack problems. Exploiting word-parallelism actually makes this complexity $\mathcal{O}\left(\frac{C}{w}n^2\right)$ for w-bit machines. On usual instances, C is small and this complexity collapses back to $\mathcal{O}(n^2)$. Our algorithm generates explanations for lazy clause generation solvers. With our new knapsack augmented overload check, we are able to solve all but 5 instances of the Pack benchmark. By performing a small transformation of the instances, we show that our method is more robust than known pre-solving methods that perform well on the original benchmark.

2012 ACM Subject Classification Computing methodologies → Planning and scheduling; Theory of computation → Constraint and logic programming

Keywords and phrases Scheduling, Cumulative constraint, Knapsacks, Overload check.

Digital Object Identifier 10.4230/LIPIcs.CP.2026.3

Supplementary Material *Software (Source Code, Instances, and Results)*: `https://github.com/Samclou/chuffed/releases/tag/KAOC-cp2026`

1 Introduction

The Cumulative constraint [1] asserts that a schedule of tasks with individual resource consumptions never overloads a resource with a given capacity. Although it finds natural applications in scheduling problems, such as assembly line balancing problems [16], it is also used to solve cutting and packing problems for scrap minimization [5]. The Resource-Constrained Project Scheduling Problem (RCPSP) consists of scheduling tasks under both resource and precedence constraints. Constraint programming, with the Cumulative constraint, shows notable success in solving this problem [24]. Enhancements to the algorithms used to filter the Cumulative constraint may lead to better solutions in an industrial context.

We present a checker for the Cumulative constraint, based on the overload check, that is augmented by reasoning on the consumptions as integral quantities. We show how general explanations for this checker can be generated in a lazy clause generation solver.

Section 2 presents background information spanning from cumulative scheduling and filtering rules for the Cumulative constraint to lazy clause generation, as well as other works that consider the integrality of resource consumptions. Section 3 defines our new checker, while Section 4 presents its algorithm. Section 5 describes how the explanations are generated in a lazy clause generation solver. Section 6 presents the experiments that evaluate our new checker, while Section 7 discusses the results obtained.

2 Background

2.1 Cumulative Scheduling

Let $\mathcal{I}$ be a set of n tasks, and let p_i and c_i for $i \in \mathcal{I}$ be the task's processing time and resource usage. Let S_i be its starting time variable. Each task i has an earliest (latest) start time noted est_i (lst_i). These values are associated with the current bounds of the domain of S_i, i.e., the set of values the variable can take. The earliest (latest) completion times of i are defined by $\text{ect}_i = \text{est}_i + p_i$ ($\text{lct}_i = \text{lst}_i + p_i$). The *execution window* of a task i is $[S_i, S_i + p_i)$, where $S_i + p_i$ represents the ending time of the task. All tasks must be scheduled into the project's *horizon*, i.e., the time interval considered $[0, t^{\max})$. However, the tasks that may be scheduled concurrently are restricted by the capacity C of the resource and their individual consumptions c_i. The Cumulative$([S_1, \ldots, S_n], [p_1, \ldots, p_n], [c_1, \ldots, c_n], C)$ constraint asserts that the total consumption from tasks in execution never, at any time, overloads a resource of capacity C [1]. Formally, for every time $t \in [0, t^{\max})$, $\sum_{i \in \mathcal{I} | t \in [S_i, S_i + p_i)} c_i \leq C$. In the context of multi-resource problems, each resource is associated with its own Cumulative constraint.

2.2 Filtering Rules

There exist many filtering algorithms designed to prune the domains of the starting time variables subject to a Cumulative constraint. They may either detect inconsistencies or filter out from the domains the values that cannot be part of a solution.

2.2.1 The Time Tabling Rule

The interval $[\text{lst}_i, \text{ect}_i)$, when non-empty, corresponds to a period during which task i must be in execution. At that time, the task must have started but cannot yet have finished. This interval, if non-empty, is called the *compulsory part* of i. By aggregating the compulsory parts of different tasks, one creates the *consumption profile* of the resource, i.e., a lower bound at each instant of the resource consumption [2].

The time tabling rule reasons on the consumption profile in two ways: 1) it detects a conflict if the resource is overloaded; 2) if a task overloads the resource at time t, then it must either start after or finish before t. Let $f(\Theta, t)$ be the consumption profile of a resource at time t given the set of tasks Θ. For brevity, for any $\Theta \subseteq \mathcal{I}$, we note the set $\mathcal{I} \setminus \Theta$ as Θ^c.

$$f(\Theta, t) = \sum_{i \in \Theta | t \in [\text{lst}_i, \text{ect}_i)} c_i$$

The checking and filtering rules can be expressed as:

$$\forall t \in [0, t^{\max}) : f(\mathcal{I}, t) > C \implies \text{conflict} \qquad \text{(TTCheck)}$$

$$\forall i \in \mathcal{I}, t \in [0, t^{\max}) : \text{ect}_i > t \wedge c_i + f(\{i\}^c, t) > C \implies \text{est}'_i > t \qquad \text{(TTFilter)}$$

The best theoretical complexity amongst propagators applying these rules is $\mathcal{O}(n)$ [10]. There are also competitive implementations with complexity in $\mathcal{O}(n^2)$ [10] and $\mathcal{O}(n \log n)$ [20].

2.2.2 The Overload Check

One can reason about intervals that must contain the entire execution of a subset of tasks. The energy of a task $i \in \mathcal{I}$ is defined by $e_i = c_i \times p_i$. The *overload check* asserts that for any subset of tasks, the energy available in the interval containing all their executions must be at

least the combined energy they require [27]. Let $\text{est}_\Theta := \min_{i\in\Theta} \text{est}_i$, $\text{lct}_\Theta := \max_{i\in\Theta} \text{lct}_i$, and $e_\Theta := \sum_{i\in\Theta} e_i$. Then, the overload check can be expressed as:

$$\forall\Theta \subseteq \mathcal{I} : e_\Theta > C(\text{lct}_\Theta - \text{est}_\Theta) \implies \text{conflict} \tag{OC}$$

It is, however, unnecessary to verify all subsets of tasks. Indeed, for fixed values of lct_Θ and est_Θ, there is no advantage in not including a task i such that $\text{est}_\Theta \leq \text{est}_i \wedge \text{lct}_i \leq \text{lct}_\Theta$ in the subset. Let $\mathcal{I}(a,b) := \{i \in \mathcal{I} \mid a \leq \text{est}_i \wedge \text{lct}_i \leq b\}$, $EST := \{\text{est}_i \mid i \in \mathcal{I}\}$, and $LCT := \{\text{lct}_i \mid i \in \mathcal{I}\}$. Thus, the rule can be equivalently expressed as:

$$\forall a \in EST, b \in LCT : e_{\mathcal{I}(a,b)} > C(b-a) \implies \text{conflict} \tag{OC'}$$

The (OC') rule is applied by propagators with time complexities in $\mathcal{O}(n)$ and $\mathcal{O}(n \log n)$ [8, 27].

2.2.3 The Time Table Overload Check

Rule (OC') can be enhanced by increasing the required energy $e_{\mathcal{I}(a,b)}$ with the consumption profile of the tasks outside of the tested set [25]. Let the compulsory energy consumption from tasks not in $\mathcal{I}(a,b)$ on the interval $[a,b)$ be $F(a,b) = \sum_{t\in[a,b)} f(\mathcal{I}(a,b)^c, t)$. The *time table overload check* is expressed as:

$$\forall a \in EST, b \in LCT : e_{\mathcal{I}(a,b)} + F(a,b) > C(b-a) \implies \text{conflict} \tag{TTOC}$$

Propagators applying the (TTOC) rule can also detect a conflict should the profile overload the resource at any point. The (TTOC) rule has a propagator with complexity in $\mathcal{O}(n^2)$ [25].

2.2.4 The Horizontally Elastic Overload Check

In the previous rules, the available energy is the capacity multiplied by the length of the interval. This bound considers tasks as *fully elastic*, i.e., tasks consume their energy anywhere in the interval with no respect to their shape (duration and resource consumption) or their individual est and lct. In Example 1, we see how this approximation leads to problems. To better approximate the available energy, one considers the tasks to be only *horizontally elastic*. When computing the available energy, the available resource at each time t is bounded by the total consumption of tasks i such that $t \in [\text{est}_i, \text{lct}_i)$ [12]. Also, the energetic consumption of a task cannot happen before the earliest starting time of that task.

▶ **Example 1.** Let $C = 2$ and $\mathcal{I} = \{1, 2, 3\}$. Every task $i \in \mathcal{I}$ starts at 0 ($\text{est}_i = \text{lct}_i - p_i = 0$), has a consumption $c_i = 1$, and processing time $p_i = i$. The resource trivially overloads at time 0. Rule (OC') is unable to detect this because the available energy $2 \times (3-0)$ considers that task 3 could use 2 units of resource at time 2 (rather than 1) and prevent the overload.

Given the tasks $\Theta \subseteq \mathcal{I}$, the amount of resource available at time t, noted $h_{\max}(\Theta, t)$, is defined by $h_{\max}(\Theta,t) = \min\left(\sum_{i\in\Theta|\text{est}_i\leq t<\text{lct}_i} c_i, C\right)$. The resource required at time t when all tasks of Θ are scheduled at their earliest is defined as $h_{\text{req}}(\Theta,t) = \sum_{\{i\in\Theta|\text{est}_i\leq t<\text{ect}_i\}} c_i$. Finally, $ov(\Theta,t)$ is the overflow at t, i.e., energy that must be assigned after time t. It is computed with the following recursive formula:

$$ov(\Theta,t) = \begin{cases} 0 & \text{if } t < \text{est}_\Theta \\ \max(0, ov(\Theta,t-1) + h_{\text{req}}(\Theta,t) - h_{\max}(\Theta,t)) & \text{otherwise} \end{cases}$$

In other words, the energy that must be assigned after time t is the one that must be assigned after time $t-1$, plus the resource required at time t, minus the resource used at time t. Taking the maximum with 0 corrects the case where $h_{\max}(\Theta,t)$ is over the resource used.

With this, the *horizontally elastic overload check* can be expressed as:

$$\forall b \in LCT : ov(\mathcal{I}(-\infty, b), b-1) > 0 \implies \text{conflict} \tag{HEOC}$$

The (HEOC) rule has a propagator with complexity in $\mathcal{O}(n^2)$ [12]

2.2.5 The Time Table Horizontally Elastic Overload Check

While Example 1 illustrates that the (OC') rule is insufficient to verify the validity of a solution, neither rule (TTOC) nor rule (HEOC) shares this weakness. Also, the enhancements of both these rules can be applied simultaneously [13]. To do so, $h_{\max}(\Theta, t)$ and $h_{\text{req}}(\Theta, t)$ are respectively redefined as follows:

$$h^{TT}_{\max}(\Theta, t) = \min\left(\sum_{i \in \Theta | t \in [\text{est}_i, \text{lct}_i) \setminus [\text{lst}_i, \text{ect}_i)} c_i, C - f(\mathcal{I}, t)\right)$$
$$h^{TT}_{\text{req}}(\Theta, t) = \sum_{i \in \Theta | \text{est}_i \leq t < \min(\text{lst}_i, \text{ect}_i)} c_i.$$

The available resource $h^{TT}_{\max}(\Theta, t)$ considers tasks that may run at time t but do not have a compulsory time at t. Any resource used by the consumption profile is considered unavailable. Similarly, the required resource $h^{TT}_{\text{req}}(\Theta, t)$ is the consumption of the tasks when scheduled at their earliest while ignoring their compulsory part. The formula for $ov^{TT}(\Theta, t)$ uses these new definitions giving the *time table horizontally elastic overload check*:

$$\forall b \in LCT : ov^{TT}(\mathcal{I}(-\infty, b), b-1) > 0 \implies \text{conflict} \tag{TTHEOC}$$

This formulation differs from the one presented in the original work [13], but is equivalent.

The (TTHEOC) dominates all variations of the overload check presented in this overview and is applied in time $\mathcal{O}(n^2)$ [13].

2.3 Reasoning on Integral Consumptions

It can be worthwhile to extract information from the resource consumptions. As an example, the profile can be used to dynamically detect disjunctive pairs of tasks, i.e., tasks whose executions cannot overlap [11]. This leads to filtering rules that are not dominated by other known rules. However, few rules directly reason about the integrity of the consumptions. Filtering rules seldom evaluate the actual available resource, at a specific time point, given the resolution state. The (HEOC) rule rather approximates this value to $h_{\max}$, the minimum between the capacity and the sum of all potential consumptions. Recent work augments the energetic reasoning rule (a strong rule that dominates the (TTOC) rule) through the resolution of a tri-partition problem [3]. By computing the maximum amount of energy that tasks can use outside a given time interval, they can better determine the energy required in that interval, which strengthens the filtering. The sub-resolutions of the tri-partition problems with dynamic programming lead to a pseudo-polynomial complexity in $\mathcal{O}(C^2 n^3)$.

In the context of pre-solving, more precisely coefficient strengthening, various rules are used to increase the relative consumption of tasks and decrease the resource capacity before solving the RCPSP [23]. These changes cannot affect the solution space of the problem; they may only make the resource constraints easier to filter. These rules are as follows:

- Any consumption c_i greater than $C - \min_{j \in \mathcal{I}} c_j$ can be increased to C.
- Let $d = \gcd(\{c_j \mid j \in \mathcal{I} \wedge 0 < c_j < C\})$. Then, all consumption c_i, as well as C, can respectively be reduced to $\left\lfloor \frac{c_i}{d} \right\rfloor$ and $\left\lfloor \frac{C}{d} \right\rfloor$.

- Let $knapsack(\Theta, C) = \max\left\{\sum_{i\in\Theta} c_i x_i \mid \sum_{i\in\Theta} c_i x_i \leq C, x_i \in \{0,1\}\right\}$ note the *maximum available resource* for the subset of tasks Θ with a resource capacity of C. For a given time t, the relevant set Θ to consider changes. Let the upper bound on the maximum available resource, at any time, for tasks that do not completely use the resource be $\kappa = \max(\{1\} \cup \{knapsack(\{j \in \mathcal{I} \mid t \in [\text{est}_j, \text{lct}_j) \wedge c_j < C\}, C) \mid t \in [0, t^{\max})\})$. Then, all consumption c_i such that $c_i = C$, as well as C, can be reduced to κ.
- Let $P(i) \subseteq \{i\}^c$ be the set of tasks that are linked to task $i \in \mathcal{I}$ by precedence constraints. Then, all consumption c_i such that $0 < c_i < C$ can be increased in order to fill the gap between C and the maximal consumption from tasks that can be concurrent to i, i.e., c_i is increased to $C - knapsack(\{j \in \{i\}^c \mid [\text{est}_i, \text{lct}_i) \cap [\text{est}_j, \text{lct}_j) \neq \emptyset\} \setminus P(i), C - c_i)$.

These rules are applied once before launching the solver. They are applied in pseudo-polynomial time since multiple knapsack problems are solved using dynamic programming. This coefficient strengthening only helps energy-based filtering rules. Time tabling is unaffected since any propagation detected after the pre-solving is detectable beforehand.

2.4 Lazy Clause Generation

Lazy clause generation solvers couple a CP solver with a SAT solver. Any integral variable X is associated with literals, i.e., Boolean variables that are kept consistent with the domain of X. For example, the domain of S_i given by $[\text{est}_i, \text{lst}_i]$ is encoded with literals $[\![\text{est}_i \leq S_i]\!]$ and $\neg[\![\text{lst}_i + 1 \leq S_i]\!]$, both set to true. We use $[\![X \leq v]\!]$ as a shorthand for $\neg[\![v + 1 \leq X]\!]$.

Whenever solvers propagate a constraint, that propagation is *explained*. For instance, a conflict detected by rule (TTCheck) at time t for task set Θ may be explained as follows:

$$\bigwedge_{i\in\Theta} ([\![\text{est}_i \leq S_i]\!] \wedge [\![S_i \leq \text{lst}_i]\!]) \rightarrow \text{False}$$

This means that the domains of the starting time variables of the tasks in Θ lead to a conflict. Similarly, filtering the bounds of the domain of a variable also creates an explanation. The "False" conclusion of the previous explanation is replaced by the literal associated with the domain change, i.e., $[\![y \leq X]\!]$ if the lower bound of X is filtered to y. These explanations are used to learn *nogoods* that prevent similar conflicts [9, 19].

In order to learn useful and reusable nogoods, the explanations must not be too specific. A first way to generalise an explanation is to reduce the number of literals. When applied to the (TTCheck) rule, the explanation should be generated with a minimal Θ, i.e., $\forall\Omega \subset \Theta,\ f(\Omega, t) \leq C$. A second way is to *lift* the constants used in the literals, making the explanation apply to domains that supersets the ones for which the propagation was detected. With rule (TTCheck), should the conflict be detected at time t with all tasks in Θ fixed ($\text{est}_i = \text{lst}_i\ \forall i \in \Theta$), it is not necessary for each task to start at the instant they are currently scheduled for the conflict to hold. It is sufficient that they have a compulsory part at time t. The explanation can then be generalised as follows:

$$\bigwedge_{i\in\Theta} ([\![t - p_i - 1 \leq S_i]\!] \wedge [\![S_i \leq t]\!]) \rightarrow \text{False} \qquad \text{(TTExpl)}$$

This last explanation, in the context of the (TTCheck) rule, is called the *pointwise explanation* [24]. Efficient propagators of this rule reason on intervals of time for which the consumption profile is constant. In that context, the value of t used to generate the pointwise explanation is chosen as the midpoint of the first interval for which the conflict is detected.

3 Better Bounds on the Resource Availability

Our main contribution is an enhancement to the (TTHEOC) rule by defining a stronger evaluation of the resource availability.

▶ **Example 2.** Consider $C = 3$ and $\mathcal{I} = \{1, 2, 3, 4\}$ where, $\forall i \in \mathcal{I}$, $\text{est}_i = 0$, $\text{lct}_i = 7$, $p_i = 2$, and $c_i = 2$. When we apply the standard (OC') rule with $\Theta = \mathcal{I}(0, 7) = \mathcal{I}$, the available energy is $3 \times (7 - 0) = 21$. Since $e_\Theta = 16$, no conflict is detected. This does not change if we instead apply rule (TTHEOC) or the augmented energetic reasoning mentioned in Section 2.3. However, a parity test shows that the third unit of resource is never used, the available energy can be reduced to $2 \times (7 - 0) = 14$, which permits the detection of a conflict.

Example 2 illustrates a case in which the resource available to a set of tasks can be better approximated based on divisibility. However, there are cases that such a basic reasoning misses. It is why we consider the quantity of resource still available to the subset of tasks Θ at time t, not counting what is already used by compulsory parts, to be the solution of a knapsack problem. Let $h^K_{\max}(\Theta, t)$ be that quantity:

$$h^K_{\max}(\Theta, t) := knapsack(\{i \in \Theta \mid t \in [\text{est}_i, \text{lct}_i) \setminus [\text{lst}_i, \text{ect}_i)\}, C - f(\mathcal{I}, t))$$

Here, only the tasks in Θ that may execute at time t, but are not forced to, are considered. This ensures that compulsory parts do not contribute to the resource availability, as it was the case with $h^{TT}_{\max}(\Theta, t)$. We define $h^K_{\text{req}}(\Theta, t)$ like $h^{TT}_{\text{req}}(\Theta, t)$:

$$h^K_{\text{req}}(\Theta, t) = h^{TT}_{\text{req}}(\Theta, t) = \textstyle\sum_{i \in \Theta \mid \text{est}_i \leq t < \min(\text{lst}_i, \text{ect}_i)} c_i$$

Our new overflow $ov^K(\Theta, t)$ is still computed with the same formula, but it uses the new functions $h^K_{\max}(\Theta, t)$ and $h^K_{\text{req}}(\Theta, t)$:

$$ov^K(\Theta, t) = \begin{cases} 0 & \text{if } t < \text{est}_\Theta \\ \max(0, ov^K(\Theta, t-1) + h^K_{\text{req}}(\Theta, t) - h^K_{\max}(\Theta, t)) & \text{otherwise} \end{cases}$$

Our new *knapsack augmented overload check* is expressed as follows:

$$\forall b \in LCT : ov^K(\mathcal{I}(-\infty, b), b - 1) > 0 \implies \text{conflict} \qquad \text{(KAOC)}$$

Note that since $h^K_{\max}(\Theta, t) \leq h^{TT}_{\max}(\Theta, t)$, our (KAOC) rule dominates the (TTHEOC) rule.

4 Propagation Algorithms

This section describes the algorithms and data structures used to apply the (KAOC) rule.

4.1 Conflict Detection

To test the sets $\mathcal{I}(-\infty, b)$ for all $b \in LCT$, we consider an initially empty set Θ that we augment by incrementally adding tasks i such that $\text{lct}_i = \min\{\text{lct}_j \mid j \in \Theta^c\}$. Applying the (KAOC) rule using the recursion would necessitate considering each individual time value, which is inefficient. Thus, we find time intervals on which profile heights, required resources, and resource availabilities are constant, allowing us to iterate over multiple time values in a single step. We use a doubly linked list that we call *Profile* to store the time points. Since profile heights, required resources, and resource availability can only vary at the est, lst, ect, or lct of a task, the Profile has one time point for each distinct such value,

for a total of at most $4n+1$ time points (counting a tail sentinel). A time point tp has these attributes: the *time* of the beginning of the interval, its profile height $f(\mathcal{I}, tp.time)$, its bitset, and its required resource $h^K_{\text{req}}(\Theta, tp.time)$. The bitset is used to compute $h^K_{\max}(\Theta, tp.time)$, as explained in Section 4.2.

Algorithm 1 KnapsackAugmentedScheduleTasks(Θ, C)

```
1 tp ← Profile.find(est_Θ); ov ← 0;
2 while tp.time ≠ lct_Θ do
3     d ← tp.next.time − tp.time;
4     h_max ← GetAvailableResource(tp.bitset, tp.f, C);
5     h_req ← tp.req;
6     ov ← max(0, ov + d × (h_req − h_max));
7     tp ← tp.next;
8 return ov;
```

Algorithm 1 tests whether rule (KAOC) detects a conflict with the set Θ. It initialises the overflow to 0, then iterates over all time points whose time is in $[\text{est}_\Theta, \text{lct}_\Theta)$. The first relevant time point is found at line 1 with the function Profile.find(est_Θ). Line 3 identifies the duration of the time interval starting at the current time point. Line 4 computes $h^K_{\max}(\Theta, tp.time)$ with Algorithm 4, using the time point's bitset and profile height. Line 6 updates the overflow to $ov^K(\Theta, tp.time + d - 1)$. The formula used differs from the definition in Section 3 because we deal with intervals rather than individual units of time. The algorithm returns $ov^K(\Theta, \text{lct}_\Theta - 1)$. Algorithm 1 is very similar to the one originally presented for the (HEOC) rule [12]. The main differences are at lines 4-5, where this version finds $h^K_{\max}(\Theta, tp.time)$ and $h^K_{\text{req}}(\Theta, tp.time)$ rather than $h_{\max}(\Theta, tp.time)$ and $h_{\text{req}}(\Theta, tp.time)$. All other differences are simplifications due to our new version not adapting the horizontally elastic edge finding rule presented in the original paper. Since each of the $\mathcal{O}(n)$ time points is processed in $\mathcal{O}(C)$ time (see Section 4.2), Algorithm 1 runs in $\mathcal{O}(Cn)$ time.

Algorithm 2 KnapsackAugmentedOverloadCheck($\mathcal{I}$, C)

```
1 Θ ← ∅;
2 Sort est, ect, lst, and lct in non-decreasing order;
3 Initialize the Profile;
4 if ∃tp ∈ Profile : f(I, tp.time) > C then fail;
5 for i ∈ I in non-decreasing order of lct_i do
6     Θ ← Θ ∪ {i};
7     Profile.addTask(i);
8     if KnapsackAugmentedScheduleTask(Θ, C) > 0 then fail;
```

Algorithm 2 applies the (KAOC) rule by calling Algorithm 1 with all relevant subsets of tasks. Rule (KAOC) detects a conflict when Algorithm 1 returns a positive overflow. We initialise the Profile by sorting the est, lst, ect, and lct values, then computing the profile heights for all time points in $\mathcal{O}(n \log(n))$. All bitsets are initialised (each in $\mathcal{O}(C)$ time) and each required resource is set to 0. Line 7 updates each time point tp as follows:

- As described in Section 4.2, the task's impact is added to *tp.bitset* with Algorithm 3, if $tp.time \in [\text{est}_i, \text{lct}_i) \setminus [\text{lst}_i, \text{ect}_i)$.
- The required resource tp.req is increased by c_i if $tp.time \in [\text{est}_i, \min(\text{ect}_i, \text{lst}_i))$.

Since updating a specific time point is done in $\mathcal{O}(C)$ time, and $\mathcal{O}(n)$ time points are updated at each inclusion, adding a task to Θ is performed in $\mathcal{O}(Cn)$ time. Since the loop iterates n times, and each iteration is performed in $\mathcal{O}(Cn)$ time, Algorithm 2 runs in $\mathcal{O}(Cn^2)$. This complexity is close to the $\mathcal{O}(n^2)$ complexity of the (TTHEOC) propagator since C is often small, while Section 4.2 shows that this factor can be divided by 64 with modern processors.

4.2 Computing the Resource Availability

With the (KAOC) rule, knowing the available resource $h^K_{\max}(\Theta, t)$ at each relevant time t requires solving multiple knapsack problems. These problems can be solved with dynamic programming using a bitset of $C+1$ bits indexed from 0 to C [21]. Initially, all bits except bit 0 are set to false. Bit x indicates whether there is a subset of the tasks currently considered with a total consumption of x. An advantage of bitsets over arrays of Booleans is that they help exploit bitwise operations on machine words. As such, on 64-bit processors, the number of operations is reduced by a factor of nearly 64 compared to using arrays.

When adding a task to the Profile at line 7 of Algorithm 2, each bit $j \in \{c_i, \dots, C\}$, in decreasing order (for each updated bitset), becomes the disjunction of itself and bit $j - c_i$. This is done by taking the disjunction between the bitset and itself left-shifted by c_i bits. To obtain the bitset consistent with the tasks in Θ at time t, starting from a bitset in its initial state, we run Algorithm 3 with the bitset for each $i \in \Theta$ such that $t \in [\text{est}_i, \text{lct}_i) \setminus [\text{lst}_i, \text{ect}_i)$. Then, calling Algorithm 4 on this bitset with the profile height $f(\mathcal{I}, t)$ returns the greatest $j \leq C - f(\mathcal{I}, t)$ for which bit j is set to true, i.e., $h^K_{\max}(\Theta, t)$. Finding the most significant bit in a machine word can be done in constant time with specific assembly operations.

Algorithm 3 AddItem(*bitset*, i)

```
1 bitset ← bitset ∨ (bitset ≪ c_i);
```

Algorithm 4 GetAvailableResource(*bitset*, f, C)

```
1 return MostSignificantBit(bitset ∧ (1 ≪ (C − f + 1)) − 1);
```

The initialisation of a bitset and these two algorithms have a time complexity of $\mathcal{O}(C)$. Therefore, computing $h^K_{\max}(\Theta, t)$ from scratch is done in $\mathcal{O}(C|\Theta|)$. This complexity can be amortised by processing the subsets of tasks in incremental order. When task i is added to Θ in Algorithm 2, Algorithm 3 is called with i on the bitset of each time point in the Profile with $t \in [\text{est}_i, \text{lct}_i) \setminus [\text{lst}_i, \text{ect}_i)$. As such, the work done to compute $h^K_{\max}(\Theta \setminus \{i\}, t)$ at the previous iteration is reused to compute $h^K_{\max}(\Theta, t)$.

5 Explanation Generation

To the best of our knowledge, no explanation generalisation scheme has been published for either rules (HEOC) or (TTHEOC). We present a way to generate general explanations for conflicts detected by our new (KAOC) rule. Explanations for the (HEOC) and (TTHEOC) rules are obtained by simplifying our method.

Conflicts found at line 4 of Algorithm 2 are explained as for the (TTCheck) rule with the pointwise explanation (TTExpl). From now on, we consider that Algorithm 2 finds a

conflict at line 8 given Θ. A naive explanation is given as follows:

$$\bigwedge_{i \in \Theta \cup \{j \in \mathcal{I} | [\text{lst}_j, \text{ect}_j) \cap [\text{est}_\Theta, \text{lct}_\Theta) \neq \emptyset\}} ([\![\text{est}_i \leq S_i]\!] \wedge [\![S_i \leq \text{lst}_i]\!]) \rightarrow \text{False}$$

This explanation has two flaws. First, it assumes that all tasks in Θ or with a compulsory part in the interval $[\text{est}_\Theta, \text{lct}_\Theta)$ are relevant to the conflict, which is often false. Second, only using the current bounds of the variables misses the possibility of deducing the conflict on supersets of the domains, which was a strength of the pointwise explanations for the (TTCheck) rule [24]. The following subsections address these issues.

5.1 Set Reduction

Figure 1 illustrates a conflict detected by (KAOC). If we pretend it illustrates only a late slice of the schedule, Θ will contain many tasks of smaller lct that are irrelevant for the conflict, but that are part of the naive explanation. These unneeded tasks should be removed from the explanation since their presence may impede the nogood from preventing the conflict. Indeed, any change to the state of these tasks will satisfy the nogood and stop it from propagating on the relevant tasks. This section presents a set reduction technique performing this removal.

Algorithm 5 SetReduction(Θ, C)

```
1  Ω ← Θ;
2  for i ∈ Ω in non-decreasing order of est_i do
3      if est_i = lst_i then Θ ← Θ \ {i}; continue ;
4      tp ← Profile.find(est_i);
5      while tp.time ≠ lct_i do
6          if tp.time ∉ [lst_i, ect_i) then
7              tp.bitset.reset();
8              for j ∈ Θ \ {i} do
9                  if tp.time ∈ [est_j, lct_j) \ [lst_j, ect_j) then AddItem(tp.bitset, j) ;
10             if tp.time < min(lst_i, ect_i) then tp.req ← tp.req − c_i;
11         tp ← tp.next;
12     ov ← KnapsackAugmentedScheduleTask(Θ \ {i}, C);
13     if ov > 0 then Θ ← Θ \ {i}; continue ;
14     tp ← Profile.find(est_i);
15     while tp.time ≠ lct_i do
16         if tp.time ∉ [lst_i, ect_i) then
17             AddItem(tp.bitset, i);
18             if tp.time < min(lst_i, ect_i) then tp.req ← tp.req + c_i ;
19         tp ← tp.next;
20 return Θ;
```

Algorithm 5 performs the set reduction. It greedily tests, for each task $i \in \Theta$, whether its removal from Θ prevents the detection of the conflict by our (KAOC) rule, i.e., if $ov^K(\Theta \setminus \{i\}, \text{lct}_{\Theta \setminus \{i\}} - 1) \leq 0$. This is done by the main loop at line 2, which also maintains the consistency of the Profile with respect to the current state of Θ. Let i be the task currently tested in the loop. When $\text{est}_i = \text{lst}_i$, the task is fixed and its contribution to the conflict can only come from its compulsory part. Then, i can be freely removed from

Θ on line 3 and no update to the Profile is necessary. Otherwise, an actual test must be performed by removing the impact of i from the Profile. At line 4, we find the time point with time est_i, which is the first time point that may be impacted by task i. Note that the modification of a bitset by Algorithm 3 is irreversible (two distinct bitset states followed by a call to Algorithm 3 with a specific task may lead to a unique bitset state). Thus, the bitsets for which Algorithm 3 was called with i must be completely recomputed for $\Theta \setminus \{i\}$. The required resource $h^K_{req}(\Theta, t)$ must also be decreased by c_i for each time point such that $t \in [\text{est}_i, \min(\text{lst}_i, \text{ect}_i))$. The loop at line 5 modifies each relevant time point. The re-computation of the bitsets is done on lines 7-9, while the correction of the required resource is done at line 10. A call to Algorithm 1 at line 12 determines if i can be removed. In that case, we can immediately consider the removal of the next task. Otherwise, the Profile is reverted to its previous state at lines 15-19, since task i must remain in Θ.

Given that the modification of a time point is done in $\mathcal{O}(Cn)$ time, that only $\mathcal{O}(n)$ time points are modified for each tested task, and that only $\mathcal{O}(n)$ tasks are tested, Algorithm 5 runs in $\mathcal{O}(Cn^3)$ time. While this time complexity is rather substantial, it should be noted that Algorithm 5 only runs when a conflict is found. The argument justifying this complexity also considers that Θ contains many unfixed tasks (fixed tasks are each removed in $\mathcal{O}(1)$ time). As such, this worst-case complexity can only be reached when an exponentially large branch of the search tree is pruned, which is more reasonable.

This method does not directly remove from the explanation the tasks with a compulsory part on $[\text{est}_\Theta, \text{lct}_\Theta)$ that are not in Θ. First, reducing Θ can shrink $[\text{est}_\Theta, \text{lct}_\Theta)$ and indirectly remove such tasks from the explanation. Second, the next section discusses value lifting, a technique that relaxes the domain bounds, which can shrink and remove compulsory parts, thus removing the associated tasks from the explanation when possible. Value lifting alone can never remove a task in Θ from the explanation, justifying our set reduction.

5.2 Value Lifting

While the previous technique aims to generalise the explanation by removing tasks that do not participate in the conflict, value lifting rather relaxes the domain bounds that are unnecessarily tight. To do this, we attempt to decrease (increase) the est (lst) of each task, as well as the respective completion times, and see if the conflict still holds. For tasks that are in the explanation but not in Θ, the domain relaxation may remove the intersection between their compulsory part and $[\text{est}_\Theta, \text{lct}_\Theta)$. Then, these tasks are removed from the explanation.

Algorithm 6 lifts the earliest starting time of a task that contributes to the conflict. A sketch trace of this algorithm is given in Example 3. The idea is to incrementally and simultaneously reduce est_i and ect_i until reaching the previous time point on the Profile, then test whether the conflict is still detected. We keep reducing these values until the conflict no longer exists, or a minimal value is reached. Line 1 finds a time beyond which we stop lifting the value. In our case, we take the maximum between the time of the first time point, i.e., $\min(EST)$, and the initial lower bound of S_i when solving starts. Then, line 3 finds the latest time points with a time earlier than est_i and ect_i, respectively. Note that the function Profile.findBefore(est_i) finds the latest time point with time $t < \text{est}_i$, even when the current value of est_i is associated with no time point. The main loop at line 4 incrementally pushes those time points to the left to lift the earliest starting time. Line 5 finds the step length necessary to reach the previous time point. Since the time points are not necessarily equidistant, at most one of $\text{est}_i - \mathit{step}$ and $\text{ect}_i - \mathit{step}$ may not currently be associated with a time point. In order to keep the Profile consistent, a virtual temporary time point is added at that unrepresented time as a copy of the previous time point at lines 7 or 10. In any case,

the Profile is kept consistent by updating the time points at times $\text{est}_i - step$ and $\text{ect}_i - step$ at lines 7, 8, 10, and 11 by calling Algorithm 7. Algorithm 7 modifies the required resource, the profile height, and/or the bitset of a time point depending on the case at hand. The local modification of two time points in the Profile is sufficient to simulate the case where task i can start *step* units of time earlier than it currently can. A call to Algorithm 1 at line 12 detects whether the conflict still holds when lifting est_i by *step*. If so, the virtual time point is removed at lines 14 or 16 (if one was added). If the time point associated with the lifting already existed, the current time point becomes its predecessor at lines 15 or 17. Then, we update the est_i and ect_i to be decreased by *step* at line 18, and can continue to the next iteration of the loop. If the conflict no longer holds, line 19 reverts all changes made to the Profile in the current iteration of the loop and the loop ends. The reversion corresponds to decreasing/increasing all integers that were changed, reverting the modified bitsets to their saved state, and potentially removing a virtual time point. At the end of the loop, a time point is added in the Profile if no time point already corresponds to the new est_i or ect_i.

Algorithm 6 LiftEst(i, Θ, C)

```
1  min_est ← max(Profile.first.time, init_est_i);
2  if est_i = min_est then return;
3  tp ← Profile.findBefore(est_i); tp' ← Profile.findBefore(ect_i);
4  do
5      step ← min(est_i − tp.time, ect_i − tp'.time);
6      if est_i − step > tp.time then
7          tp.insertAfter(est_i − step); AdaptTimePoint(tp.next, i, Θ, est);
8      else tp.bitset.save(); AdaptTimePoint(tp, i, Θ, est);
9      if ect_i − step > tp.time then
10         tp'.insertAfter(est_i − step); AdaptTimePoint(tp'.next, i, Θ, ect);
11     else tp'.bitset.save(); AdaptTimePoint(tp', i, Θ, ect);
12     ov ← KnapsackAugmentedScheduleTasks(Θ, C);
13     if ov > 0 then
14         if est_i − step > tp.time then tp.removeAfter();
15         else tp ← tp.previous;
16         if ect_i − step > tp'.time then tp'.removeAfter();
17         else tp' ← tp'.previous;
18         est_i ← est_i − step; ect_i ← ect_i − step;
19     else revertChanges(); break;
20 while est_i > min_est;
21 if tp.next.time ≠ est_i then
22     tp.insertAfter(est_i − step); AdaptTimePoint(tp.next, i, Θ, est);
23     if tp = tp' then tp' ← tp.next;
24 if tp'.next.time ≠ ect_i then
25     tp'.insertAfter(ect_i − step); AdaptTimePoint(tp'.next, i, Θ, ect);
```

▶ **Example 3.** Suppose that Algorithm 2 detects a conflict in the situation illustrated by Figure 1 while the Profile is in the state defined by Table 1.

When lifting est_2, we use a step of 1, leading to the following changes in the Profile:

- The required resource at time 1 must be increased by c_2 (becoming 6) since task 2 now starts being available at that time.

Algorithm 7 AdaptTimePoint(tp, i, Θ, $type$)

```
1 if type ∈ {ect, lst} ∧ lst_i < ect_i then tp.f ← t.f − c_i;
2 if i ∈ Θ then
3     if type ∈ {est, lct} ∨ (type ∈ {ect, lst} ∧ lst_i < ect_i) then AddItem(tp.bitset, i);
4     if type = est ∨ (type = lst ∧ lst_i < ect_i) then tp.req ← tp.req + c_i;
5     if type = ect ∧ lst_i ≥ ect_i then tp.req ← tp.req − c_i;
```

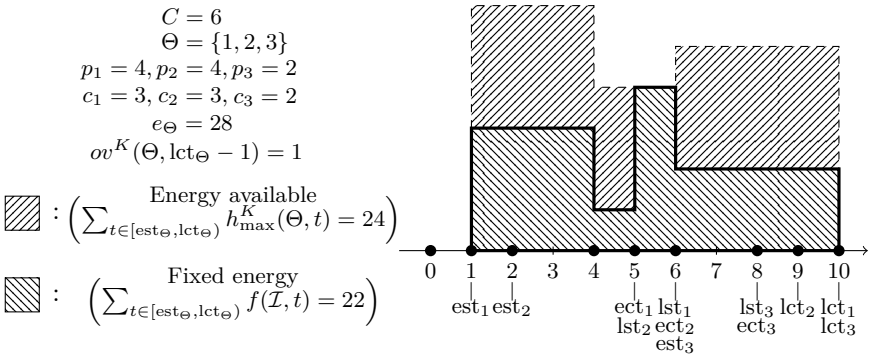


Figure 1 Visualisation of a state for which a conflict is detected by rule (KAOC). Tasks out of Θ that contribute to the consumption profile are unspecified to simplify. Dots on the time line specify times represented by a time point in the Profile.

- The impact of task 2 is added to the bitset of time 1 with Algorithm 3 for the same reason, becoming 100 1001. The previous state of the bitset is saved for quick reversion.
- The compulsory part of task 2 is removed from time 5, decreasing its profile height to 1.
- The impact of task 2 is also added to the bitset of time 5 since the task is now freely available at that time, becoming 100 1001. The previous state of the bitset is saved.

Now, the Profile is consistent with the considered lifting and a call to Algorithm 1 finds an overflow energy of 1, which means that the conflict still holds. Thus, est_2 can be decreased to 1 for the conflict explanation. The next considered step would be of 1 again and, this time, we would see that est_2 cannot be lifted beyond time 1.

The lifting of any lst_i is done symmetrically. Note that a call to Algorithm 6 adds at most one time point to the Profile. Thus, as long as we never try to lift a value that has already been lifted, the Profile counts at most $6n + 1$ time points. As such, the runtime complexity of each call to Algorithm 1 remains $\mathcal{O}(Cn)$, and the loop of Algorithm 6 iterates $\mathcal{O}(n)$ times. Also, although a call to Algorithm 3 is an irreversible operation, revertChanges() can be done in $\mathcal{O}(C)$ time since we save copies of the affected bitsets before their modification. Thus, Algorithm 6 runs in $\mathcal{O}(Cn^2)$, and lifting all values in the explanation takes $\mathcal{O}(Cn^3)$ time.

In order to generate the explanation, we first find the reduced set Θ' by calling Algorithm 5. After that, we remove from the Profile any compulsory part that does not intersect $[\text{est}_{\Theta'}, \text{lct}_{\Theta'})$. Then, for each task $i \in \Theta'^c$ such that $[\text{lst}_i, \text{ect}_i) \cap [\text{est}_\Theta, \text{lct}_\Theta) \neq \emptyset$, we run Algorithm 6 with Θ', followed by its lst-equivalent, in order to find new bounds est'_i and lst'_i. Similarly, we run Algorithm 6 on all tasks $i \in \Theta'$ to find est'_i, then repeat the same with its equivalent for lifting the lst. Finally, we explain the conflict with:

$$\bigwedge_{i \in \Theta' \cup \left\{j \in \mathcal{I} \mid [\text{lst}'_j, \text{est}'_j + p_j) \cap [\text{est}_{\Theta'}, \text{lct}_{\Theta'}) \neq \emptyset\right\}} \left([\![\text{est}'_i \leq S_i]\!] \wedge [\![S_i \leq \text{lst}'_i]\!]\right) \rightarrow \text{False}$$

time	previous	next	req	f	bitset
0	-	1	0	0	000 0001
1	0	2	3	3	000 1001
2	1	4	6	3	100 1001
4	2	5	6	1	100 1001
5	4	6	0	4	000 1001
6	5	8	2	2	110 1101
8	6	9	0	2	110 1101
9	8	10	0	2	010 1101
10	9	∞	0	0	000 0001
∞	10	-	0	0	000 0001

Table 1 Description of the Profile state associated to the conflict illustrated at Figure 1.

6 Experiments

We evaluate the performance of our new checker by solving various instances of the RCPSP. As a reminder, this problem consists of a set of tasks $\mathcal{I}$, of resources $\mathcal{R}$, and precedence relationships $\mathcal{P}$. Each resource $r \in \mathcal{R}$ is subject to a Cumulative constraint with a capacity C_r where tasks $i \in \mathcal{I}$ have a consumption of $c_{r,i}$. The precedences $(i, j) \in \mathcal{P}$ force task i to finish before the start of task j. In other words, $(i, j) \in \mathcal{P} \Rightarrow S_i + p_i \leq S_j$. The goal of this problem is to minimise the makespan, i.e., $\max_{i \in \mathcal{I}}\{S_i + p_i\}$.

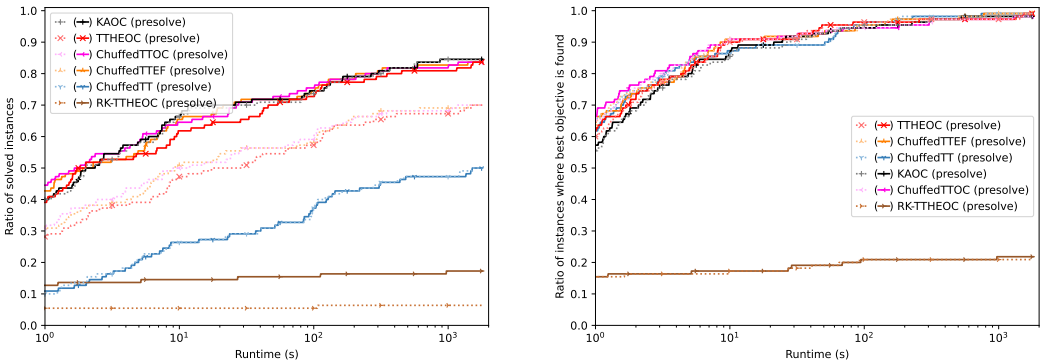
We model the problem in MiniZinc [18] and use version 0.13.0 of the Chuffed solver [6], unless specified otherwise. We compare 6 propagators of the Cumulative constraint:

- Chuffed's implementation of the time tabling propagator (noted ChuffedTT),
- Chuffed's implementation of the (TTOC) propagator (noted ChuffedTTOC),
- Chuffed's implementation of the time table edge-finding propagator (noted ChuffedTTEF),
- our implementation in Chuffed of our (KAOC) propagator (noted KAOC),
- our implementation in Chuffed of the (TTHEOC) propagator (obtained by removing the bitsets from our (KAOC) propagator, and noted TTHEOC),
- the (TTHEOC) propagator [13] implemented in Choco [22] (noted RK-TTHEOC).

All variations also include a time tabling propagator.

We use Chuffed with the "toggle-vsids" option to alternate between the static variable selection heuristic and the activity based one [17]. The option "sbps" is turned on for the value selection heuristic, which simulates a large neighbourhood search [7, 26]. Finally, the option "eager-limit" is set to 7500, which enhances the resolution of the Pack_d instances without affecting the others. With the Choco solver, we only use the static search heuristic since it is what solved the most instances with the (TTHEOC) rule in the original paper [13]. This last variation is a rough comparison since the search heuristics and the solvers' technologies differ substantially. It is, however, relevant to show the advantages of making the (TTHEOC) rule compatible with lazy clause generation solvers, which is one of our contributions.

We perform our experiments on the Pack [4], Pack_d [15] and PSPLIB [14] benchmarks. On these benchmarks, two machine words are always sufficient to encode the bitsets (the greatest C is 122). Thus Algorithm 2 effectively has a quadratic complexity on these instances. All the options are tested twice on each instance of the benchmarks: once on the base instances and a second time on the instances transformed using the pre-solving rules presented in Section 2.3. The pre-solving time is overall negligible and is not included in the runtimes. We thus compare 12 approaches. In order to show the difference in robustness between our new propagator and the pre-solving method, we introduce the new Pack+1, Pack_d+1, and PSPLIB+1 benchmarks. Each is generated, respectively, from Pack, Pack_d, and PSPLIB by adding to each instance one task of processing time 1 with a resource consumption of 1 on



(a) **(b)**

Figure 2 Pack and Pack_d benchmarks results for each method w.r.t. time. **(a)** Shows the ratio of solved instances. **(b)** Shows the ratio of instances where the best objective is found.

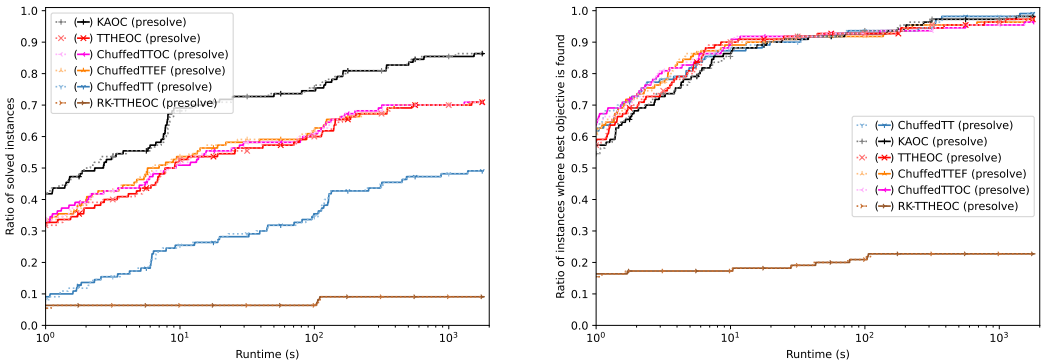
each resource. This task is subject to no precedence constraint other than those associated with the dummy tasks of the project bounds. The resolutions are run on a computer cluster equipped with AMD EPYC 7532/7502 CPUs, given a time-out of 30 minutes. Although two distinct instances may run on different computer nodes, the six resolutions of a given instance run in parallel on the same node. Our code, instances, and raw results are available[1].

7 Results

We do not show the results on the PSPLIB instances since they are rather uninteresting. All methods using the Chuffed solver perform similarly for proving optimality, while solution quality for unsolved instances degrades with stronger propagators, time tabling alone having the best performance. Pre-solving the instances also has little to no impact on this benchmark. We surmise that this is due to the disjunctive nature of the PSPLIB instances, where most filtering is probably due to the time tabling rules and precedences, rather than the energy-based reasoning. These results are also available in the code repository.

Figure 2 shows the results on the Pack and Pack_d instances. Figure 2a shows the ratio of instances solved as a function of time for each method. The resolution of the base instances is represented by dotted curves, while that of the pre-solved instances is represented by full curves. It can be seen that pre-solving the instances helps solve more instances. Indeed, excluding the methods that are not significantly affected by it, pre-solving permits the resolution of about 15 more instances out of 110. For most of these instances, an optimal solution is found very quickly and can trivially be shown to be optimal with a divisibility argument on one of their resources. These instances are, however, difficult to prove as optimal with the solver without this type of reasoning. ChuffedTT and KAOC are both mostly unaffected by pre-solving. It is unsurprising since it has been noted in Section 2.3 that pre-solving does not affect pure time tabling. Also, our new method applies an integrality reasoning similar to the one performed by the pre-solving rules, but more locally. It can be noted that all energy based methods behave similarly (with the exception of RK-TTHEOC), solving about as many instances. Nevertheless, KAOC solves instance pack040 in 2 minutes

[1] Available at: https://github.com/Samclou/chuffed/releases/tag/KAOC-cp2026



(a) **(b)**

Figure 3 Pack+1 and Pack_d+1 benchmarks results for each method w.r.t. time. **(a)** Shows the ratio of solved instances. **(b)** Shows the ratio of instances where the best objective is found.

(to our knowledge for the first time in the literature), while the other approaches fail to prove optimality before the 30 minute time-out. It should be noted that all methods using Chuffed (with and without pre-solving) performed significantly better than previously reported [23], solving about 20 more Pack instances. This is likely due to the solver options we use.

Figure 2b shows the ratio of instances for which a method found the best obtained solution as a function of the time needed to find the said solution. The RK-TTHEOC curve is again below the others, likely due to the lack of solution-based phase-saving in Choco. For the other methods, it can be seen that they manage to find similar solutions in the long run. The difficulty of the Pack and Pack_d benchmarks is mostly not to find optimal solutions, but to prove them optimal. Pre-solving has little impact on the quality of solutions found.

Figure 3 shows the results on the new Pack+1 and Pack_d+1 benchmarks. We see that the addition of the task in these benchmarks neutralised the positive effect of pre-solving. All methods are now mostly unaffected by it. This is because the global divisibility argument that previously helped solve "hard" instances no longer applies. However, KAOC's performance remains relatively unchanged, making it the dominant approach for solving these instances. The comparison between Figures 2a and 3a highlights the fragility of pre-solving and the increased robustness of our new approach. The transformation of the instances has little impact on the quality of the solutions found; it was already mostly unaffected by pre-solving.

8 Conclusion

We added reasoning based on the integrality of the resource consumptions to the horizontally elastic overload check and defined our new knapsack augmented overload check for the Cumulative constraint. We showed how this new rule can be applied in $\mathcal{O}(Cn^2)$, only adding a small pseudo-polynomial factor due to the resolution of knapsack sub-problems. We also presented how to explain our new propagator in lazy clause generation solvers. Experiments showed that the new propagator helps solve more instances for the Pack and Pack_d while being more robust than some pre-solving methods. Applying our enhancement of the overload check to the edge-finding rule represents a potential avenue for future work.

References

1 Abderrahmane Aggoun and Nicolas Beldiceanu. Extending chip in order to solve complex scheduling and placement problems. *Mathematical and Computer Modelling*, 17:57–73, 1993.

2 Nicolas Beldiceanu and Mats Carlsson. A New Multi-resource cumulatives Constraint with Negative Heights. In *Proceedings of the 8th International Conference on Principles and Practice of Constraint Programming (CP 2002)*, pages 63–79. Springer Berlin Heidelberg, 2002.

3 Jacques Carlier, Antoine Jouglet, Kristina Kumbria, and Abderrahim Sahli. More powerful energetic reasoning for the cumulative scheduling problem. *Discrete Applied Mathematics*, 378:187–209, 2026.

4 Jacques Carlier and Emmanuel Néron. On linear lower bounds for the resource constrained project scheduling problem. *European Journal of Operational Research*, 149:314–324, 2003.

5 Manuel Chastenay, Xavier Zwingmann, Claude-Guy Quimper, and Jonathan Gaudreault. Optimizing 2D Cutting: A Bin Packing Approach to Minimize Scraps and Maximize Their Reusability. In *Proceedings of the 31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*, pages 7:1–7:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025.

6 Geoffrey Chu. *Improving Combinatorial Optimization*. PhD thesis, The University of Melbourne, 2011.

7 Emir Demirović, Geoffrey Chu, and Peter J. Stuckey. Solution-Based Phase Saving for CP: A Value-Selection Heuristic to Simulate Local Search Behavior in Complete Solvers. In *Proceedings of the 24th International Conference on Principles and Practice of Constraint Programming*, pages 99–108. Springer International Publishing, 2018.

8 Hamed Fahimi, Yanick Ouellet, and Claude-Guy Quimper. Linear-time filtering algorithms for the disjunctive constraint and a quadratic filtering algorithm for the cumulative not-first not-last. *Constraints*, 23:272–293, 2018.

9 Thibaut Feydy and Peter J. Stuckey. Lazy Clause Generation Reengineered. In *Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming (CP 2009)*, pages 352–366. Springer Berlin Heidelberg, 2009.

10 Steven Gay, Renaud Hartert, and Pierre Schaus. Simple and Scalable Time-Table Filtering for the Cumulative Constraint. In *Proceedings of the 21st International Conference on Principles and Practice of Constraint Programming (CP 2015)*, pages 149–157. Springer International Publishing, 2015.

11 Steven Gay, Renaud Hartert, and Pierre Schaus. Time-Table Disjunctive Reasoning for the Cumulative Constraint. In *Integration of AI and OR Techniques in Constraint Programming*, pages 157–172. Springer International Publishing, 2015.

12 Vincent Gingras and Claude-Guy Quimper. Generalizing the Edge-Finder Rule for the Cumulative Constraint. In *Proceedings of the 25th International Joint Conference on Artificial Intelligence*, pages 3103–3109. AAAI Press, 2016.

13 Roger Kameugne, Sévérine Fetgo Betmbe, Thierry Noulamo, and Clémentin Tayou Djamegni. Improved timetable edge finder rule for cumulative constraint with profile. *Computers & Operations Research*, 172:106795, 2024.

14 Rainer Kolisch and Arno Sprecher. PSPLIB - A project scheduling problem library: OR Software - ORSEP Operations Research Software Exchange Program. *European Journal of Operational Research*, 96:205–216, 1997.

15 Oumar Koné, Christian Artigues, Pierre Lopez, and Marcel Mongeau. Event-based MILP models for resource-constrained project scheduling problems. *Computers and Operations Research*, 38:3–13, 2011.

16 Duc Anh Le, Stéphanie Roussel, and Christophe Lecoutre. Aircraft Resource-Constrained Assembly Line Balancing with Learning Effect: A Constraint Programming Approach. In *31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*, pages 25:1–25:24. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025.

17 M.W. Moskewicz, C.F. Madigan, Y. Zhao, L. Zhang, and S. Malik. Chaff: engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference*, pages 530–535, 2001.

18 Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. MiniZinc: Towards a Standard CP Modelling Language. In *Proceedings of the 13th International Conference on Principles and Practice of Constraint Programming (CP 2007)*, pages 529–543. Springer Berlin Heidelberg, 2007.

19 Olga Ohrimenko, Peter J Stuckey, and Michael Codish. Propagation via lazy clause generation. *Constraints*, 14:357–391, 2009.

20 Pierre Ouellet and Claude-Guy Quimper. Time-Table Extended-Edge-Finding for the Cumulative Constraint. In *Proceedings of the 19th International Conference on Principles and Practice of Constraint Programming (CP 2013)*, pages 562–577. Springer Berlin Heidelberg, 2013.

21 David Pisinger. Dynamic Programming on the Word RAM. *Algorithmica*, 35:128–145, 2003.

22 Charles Prud'homme and Jean-Guillaume Fages. Choco-solver: A Java library for constraint programming. *Journal of Open Source Software*, 7:4708, 2022.

23 Jens Schulz. *Hybrid Solving Techniques for Project Scheduling Problems*. PhD thesis, Technischen Universität Berlin, 2013.

24 Andreas Schutt, Thibaut Feydy, Peter J Stuckey, and Mark G Wallace. Explaining the cumulative propagator. *Constraints*, 16:250–282, 2011.

25 Petr Vilím. Timetable Edge Finding Filtering Algorithm for Discrete Cumulative Resources. In *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, pages 230–245. Springer Berlin Heidelberg, 2011.

26 Julien Vion and Sylvain Piechowiak. Une simple heuristique pour rapprocher DFS et LNS pour les COP. In *Actes des 13e Journées Francophones de la Programmation par Contraintes, JFPC 2017*, pages 39–46, 2017.

27 Armin Wolf and Gunnar Schrader. $\mathcal{O}(n \log n)$ Overload Checking for the Cumulative Constraint and Its Application. In *Declarative Programming for Knowledge Management*, pages 88–101. Springer Berlin Heidelberg, 2006.